

EC 504
Fall, 2025
Homework 6

Due Friday, November 21, 2025, 11:59 pm

Problem 6.1 (20 pts) Determine whether the statements below are true or false. Please provide a quick explanation to justify your answer.

- (a) The following three algorithms are examples of greedy algorithms: Dijkstra's shortest path algorithm, Kruskal's minimum spanning tree algorithm, Prim's minimum spanning tree algorithm.

Solution:

True. They are greedy algorithms, because they select candidates based on a local metric, and once a candidate is selected, there is no backtracking.

- (b) Bellman-Ford algorithm can be used for finding the longest path in an acyclic graph.

Solution:

This is true. Basically, take the negative of all the distances and try to find the shortest path with negative distances. Since the graph has no cycles, it has no negative weight cycles and hence Bellman Ford will find the shortest path.

- (c) In Floyd-Warshall's algorithm for finding all-pairs shortest paths, after each major outer loop k , upper bounds are computed on shortest distances $D(i, j)$ between each pair of vertices i, j . These upper bounds correspond to the shortest distance among all paths between vertices i and j which use only intermediate vertices in $1, \dots, k$.

Solution:

True. That is the proof of Floyd's algorithm.

- (d) Consider a directed, weighted graph where every arc in the graph has weight 100. For this graph, executing Dijkstra's algorithm to find a shortest path tree starting from a given vertex is equivalent to performing depth first search.

Solution:

False. When the distances are the same on every arc, Dijkstra's algorithm scans the nodes in breadth first search order (not DFS), since distance and number of hops are equivalent.

- (e) In Floyd-Warshall's algorithm, the worst case complexity does not depend on the number of edges.

Solution:

This is true. It is $O(n^3)$, where n is the number of vertices.

- (f) Floyd-Warshall's algorithm fails to converge when there are negative weight edges, but no negative weight cycles, in the graph.

Solution:

This is false; the algorithm will converge as long as there are no negative-weight cycles.

- (g) Consider a directed, capacitated graph, with origin vertex s and destination vertex t . The maximum flow which can be sent from s to t is less than or equal to the minimum of the following two quantities: the sum of the capacities of the edges leaving s , and the sum of the capacities of the edges entering t .

Solution:

This is True. Each of those numbers is the capacity of a cut in the network separating s and t , and the maximum flow must be no greater than the capacity of the minimum cut.

- (h) Consider a capacitated, directed graph, with an origin vertex s and a destination vertex t . Then, the maximum flow from vertex s to vertex t must be greater than or equal to the smallest capacity of any edge going into vertex t .

Solution:

This is false. You can have only one edge leaving vertex s with capacity 1, and only one edge entering t with capacity 10. The max flow will be 1.

- (i) In the Ford-Fulkerson algorithm with integer capacities, there is a sequence of augmenting paths that carry flow from the origin s to the destination t . The amount of flow carried by each augmenting path can be no greater than the amount of flow carried by the preceding augmenting path.

Solution:

This is false. There is no guarantee that we find the augmenting path with the maximum capacity in the first augmenting path.

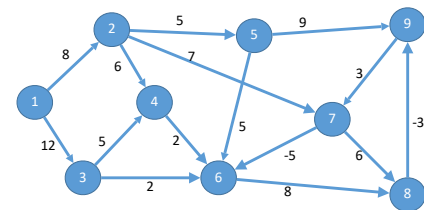
- (j) The Edmonds-Karp variation of the Ford-Fulkerson algorithm has complexity which is polynomial in the number of edges in the graph.

Solution:

This is true. Finding each augmenting path is of order number of edges, and the complexity of the Edmonds-Karp algorithm is $O(|V||E|^2)$

Problem 6.2 (20 pts)

Consider the directed weighted graph on the right. Show the distance estimates computed by each step of the Bellman-Ford algorithm for finding a shortest path tree in the graph, starting from vertex 1 to all other vertices. When scanning edges out of a vertex, consider them in clockwise order starting from straight up. Show the algorithm using a work queue to determine which vertices to scan, in order to avoid unnecessary iterations.

**Solution:**

Algorithm is shown below. Distances in brackets indicate vertices with new distances.

Initial Distances: $D(1) = 0$; $D(2) = \text{inf}$; $D(3) = \text{inf}$; $D(4) = \text{inf}$; $D(5) = \text{inf}$;
 $D(6) = \text{inf}$; $D(7) = \text{inf}$; $D(8) = \text{inf}$; $D(9) = \text{inf}$;

$Q = \{1\}$

Scan vertex 1: $D(1) = 0$; $D(2) = 8$; $D(3) = 12$; $D(4) = \text{inf}$; $D(5) = \text{inf}$;
 $D(6) = \text{inf}$; $D(7) = \text{inf}$; $D(8) = \text{inf}$; $D(9) = \text{inf}$;

```

Q = {2,3}
Scan vertex 2:      D(1) = 0; D(2) = 8; D(3) = 12; D(4) = [14]; D(5) = [13];
                    D(6) = inf; D(7) = [15]; D(8) = inf; D(9) = inf;

Q = {3,5,7,4}
Scan vertex 3:      D(1) = 0; D(2) = 8; D(3) = 12; D(4) = 14; D(5) = 13;
                    D(6) = [14]; D(7) = 15; D(8) = inf; D(9) = inf;

Q = {5,7,4,6}
Scan vertex 5:      D(1) = 0; D(2) = 8; D(3) = 12; D(4) = 14; D(5) = 13;
                    D(6) = 14; D(7) = 16; D(8) = inf; D(9) = [22];

Q = {7,4,6,9}
Scan vertex 7:      D(1) = 0; D(2) = 8; D(3) = 12; D(4) = 14; D(5) = 13;
                    D(6) = [10]; D(7) = 16; D(8) = [23]; D(9) = 22;

Q = {4,6,9,8}
Scan vertex 4:      D(1) = 0; D(2) = 8; D(3) = 12; D(4) = 14; D(5) = 13;
                    D(6) = 10; D(7) = 16; D(8) = 23; D(9) = 22;

Q = {6,9,8}
Scan vertex 6:      D(1) = 0; D(2) = 8; D(3) = 12; D(4) = 14; D(5) = 13;
                    D(6) = 10; D(7) = 16; D(8) = [18]; D(9) = 22;

Q = {9,8}
Scan vertex 9:      D(1) = 0; D(2) = 8; D(3) = 12; D(4) = 14; D(5) = 13;
                    D(6) = 10; D(7) = 16; D(8) = 18; D(9) = 22;

Q = {8}
Scan vertex 8:      D(1) = 0; D(2) = 8; D(3) = 12; D(4) = 14; D(5) = 13;
                    D(6) = 10; D(7) = 16; D(8) = 18; D(9) = [15];

Q = {9}
Scan vertex 9:      D(1) = 0; D(2) = 8; D(3) = 12; D(4) = 14; D(5) = 13;
                    D(6) = 10; D(7) = 16; D(8) = 18; D(9) = 15;

```

Done. No more vertices in Queue means no other vertices can be reached.
Shortest path tree

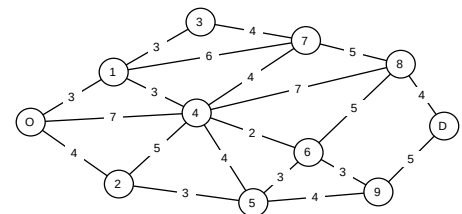
```

      7--6--8--9
      /
     2--5
    / \
   1   4
    \
     3

```

Problem 6.3 (15 pts)

Consider the weighted, undirected graph shown on the right. Assume that the edges can be traveled in both directions with the same distance. Illustrate the steps of Dijkstra's algorithm for finding a shortest path from vertex O to vertex D.



Solution:

As before, keep track of distances which change using brackets, and delete vertices already scanned from distance list

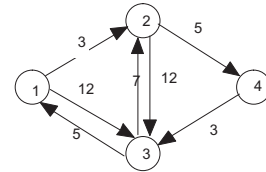
Initial Distances: $D(0)=0$; $D(1)=\text{inf}$; $D(2) = \text{inf}$; $D(3)=\text{inf}$; $D(4)=\text{inf}$; $D(5) = \text{inf}$;
 $D(6) = \text{inf}$; $D(7) = \text{inf}$; $D(8) = \text{inf}$; $D(9) = \text{inf}$; $D(D) = \text{inf}$
Scan vertex 0: $D(1)=[3]$; $D(2) = [4]$; $D(3)=\text{inf}$; $D(4)=[7]$; $D(5) = \text{inf}$;
 $D(6) = \text{inf}$; $D(7) = \text{inf}$; $D(8) = \text{inf}$; $D(9) = \text{inf}$; $D(D) = \text{inf}$

Scan vertex 1: $D(2) = 4$; $D(3)=[6]$; $D(4)=[6]$; $D(5) = \text{inf}$;
 $D(6) = \text{inf}$; $D(7) = [9]$; $D(8) = \text{inf}$; $D(9) = \text{inf}$; $D(D) = \text{inf}$
Scan vertex 2: $D(3)=6$; $D(4)=6$; $D(5) = [7]$;
 $D(6) = \text{inf}$; $D(7) = 9$; $D(8) = \text{inf}$; $D(9) = \text{inf}$; $D(D) = \text{inf}$
Scan vertex 3: $D(4)=6$; $D(5) = 7$;
 $D(6) = \text{inf}$; $D(7) = 9$; $D(8) = \text{inf}$; $D(9) = \text{inf}$; $D(D) = \text{inf}$
Scan vertex 4: $D(5) = 7$;
 $D(6) = [8]$; $D(7) = 9$; $D(8) = [14]$; $D(9) = \text{inf}$; $D(D) = \text{inf}$
Scan vertex 5: $D(6) = 8$; $D(7) = 9$; $D(8) = 14$; $D(9) = [11]$; $D(D) = \text{inf}$
Scan vertex 6: $D(7) = 9$; $D(8) = [13]$; $D(9) = 11$; $D(D) = \text{inf}$
Scan vertex 7: $D(8) = 13$; $D(9) = 11$; $D(D) = \text{inf}$
Scan vertex 9: $D(8) = 13$; $D(D) = [16]$
Scan vertex 8: $D(D) = 16$

Shortest path in reverse: D--9--5--2--0, length 16.

Problem 6.4 (10 points)

Consider the graph indicated on the right. This is a directed graph. Use the Floyd-Warshall algorithm to find the shortest distance for all pairs of vertices. Show your work as a sequence of 4 by 4 tables.



Solution:

Here is the initial table, obtained from the edges in the graph. the following tables expand the algorithm.

$D(i,j)$	$j=1$	$j=2$	$j=3$	$j=4$
$i=1$	0	3	12	inf
$i=2$	inf	0	12	5
$i=3$	5	7	0	inf
$i=4$	inf	inf	3	0

iteration: $k = 1$ as intermediate vertex

$D(i,j)$	$j=1$	$j=2$	$j=3$	$j=4$
$i=1$	0	3	12	inf
$i=2$	inf	0	12	5
$i=3$	5	7	0	inf
$i=4$	inf	inf	3	0

iteration: $k = 2$ as intermediate vertex

$D(i,j)$	$j=1$	$j=2$	$j=3$	$j=4$
$i=1$	0	3	12	[8]
$i=2$	inf	0	12	5
$i=3$	5	7	0	[12]

```

i=4      inf    inf    3    0

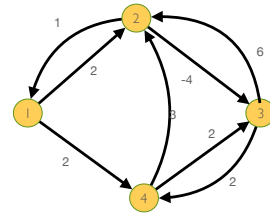
iteration: k = 3 as intermediate vertex
D(i,j)   j=1   j=2   j=3   j=4
i=1      0     3    12    8
i=2     [17]   0    12    5
i=3      5     7     0   12
i=4     [8]   [10]   3    0

iteration: k = 4 as intermediate vertex
D(i,j)   j=1   j=2   j=3   j=4
i=1      0     3   [11]   8
i=2     [13]   0    [8]   5
i=3      5     7     0   12
i=4      8    10     3    0

```

Problem 6.5 (10 points)

Consider the graph indicated on the right. This is a directed graph. Note the negative weight edge from 2 to 3. Use the first step of Johnson's algorithm, to find a reweighing of the edges such that the resulting edge weights are all non-negative. Note that you don't have to follow this with using Dijkstra's algorithm to find the shortest distance for all pairs of vertices; you just have to compute the reweighed distances.



Solution:

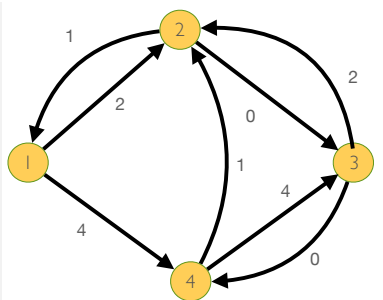
The first step in Johnson's algorithm is to add a vertex s with outgoing edges to vertices 1, 2, 3, 4, with weight 0. Then, we find the shortest distance to each vertex from s using the Bellman-Ford algorithm. Let's do that first.

```

Initial Distances:  D(s) = 0; D(1) = inf; D(2) = inf; D(3) = 0; D(4) = inf
Q = {s}
Scan vertex s:      D(1) = 0; D(2) = 0; D(3) = 0; D(4) = 0;
Q = {1,2,3,4}
Scan vertex 1:      D(1) = 0; D(2) = 0; D(3) = 0; D(4) = 0;
Q = {2,3,4}
Scan vertex 2:      D(1) = 0; D(2) = 0; D(3) = -4; D(4) = 0;
Q = {3,4}
Scan vertex 3:      D(1) = 0; D(2) = 0; D(3) = -4; D(4) = -2;
Q = {4}
Scan vertex 4:      D(1) = 0; D(2) = 0; D(3) = -4; D(4) = -2;

```

These distances now form the weights for the second part of the algorithm, where $h(1) = h(2) = 0, h(3) = -4, h(4) = -2$. The revised weights on edge (u, v) are $w_h(u, v) = w(u, v) + h(u) - h(v)$. Thus, the revised weights have $w_h(2, 3) = 0, w_h(3, 2) = 2$, and $w_h(3, 4) = 2 + -4 + 2 = 0, w_h(4, 3) = 2 + -2 + 4 = 4$, and $w_h(4, 2) = 3 - 2 = 1, w_h(1, 4) = 2 + 0 + 2 = 4$. The revised weights are shown below:



Problem 6.6 (10 pts)

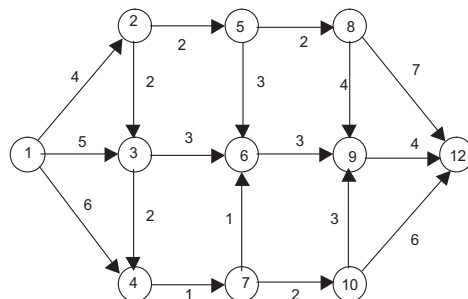
In city streets, the length of an edge often depends on the time of day. Suppose you have a directed graph of streets connecting vertices that represent intermediate destinations, and you are given the travel time on the edge as a function of the time at which you start to travel that edge. Thus, for edge e , you are given $d_e(t)$, the time it takes to travel edge e if you start at time t . These travel times must satisfy an interesting ordering property: You can't arrive earlier if you started later. That is, if $s < t$, then $s + d_e(s) \leq t + d_e(t)$. Suppose that you start at time 0 at the origin vertex 1. Describe an algorithm for computing the minimum time path to all vertices when travel times on edges are time dependent and satisfy the ordering property.

Solution:

Turns out Dijkstra's algorithm is exactly right for this problem. Dijkstra's algorithm scans vertices in the order of arrival time. The travel time on the edges for the vertex that are being scanned can be computed because we know the time at which that vertex is reached. The only difference is that the travel times on edges are computed as a function of the travel time to the origin vertex of the edge, when that vertex is permanently labeled by the algorithm.

Problem 6.7 (15 pts)

Consider the graph in the figure on the right as a directed, capacitated graph, where the numbers indicate an arc's capacity to carry flow from node 1 to node 12. In the max-flow algorithm of Ford and Fulkerson, the key step is, once a path has been found, to augment the flow and construct the residual graph for the next iteration.



- (a) Consider the path $1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 9 \rightarrow 12$. What is the capacity of this path?

Solution:

It is the minimum of the capacities of the arcs which is 2.

- (b) Suppose we send two units of the flow along path $1 \rightarrow 2 \rightarrow 5 \rightarrow 8 \rightarrow 12$. Construct the residual graph which remains after this flow has been sent. Draw this graph.

Solution:

The changes are as follows: The capacity of the arc (1,2) is reduced by 2 to 4. A reverse arc (2,1) is introduced, capacity 2. The arc (2,5) is removed and a reverse arc (5,2), capacity 2 is

introduced. The arc (5,8) is removed, and a reverse arc (8,5), capacity 2 is introduced. The arc (8,12) has reduced capacity 5, and a reverse arc, (12,8) is introduced with capacity 2. The rest of the graph does not change.

- (c) Find the maximum flow from node 1 to node 12 in this graph.

Solution:

This requires working through the Ford-Fulkerson algorithm. Total flow will be 6. You can place 2 units along 1-2-5-8-12, 3 units along 1-3-6-9-12, and 1 unit along 1-4-7-10-12. Those are all minimum hop paths. Then, there are no paths left, because one cannot cross from 1-2-3-4 to the rest of the nodes. Alternatively, note the cut of capacity 6, obtained by removing arcs 2-5, 3-6, 4-7.